AI ASSSIGNMENT-13.4

NAME:N.Vaishnavi

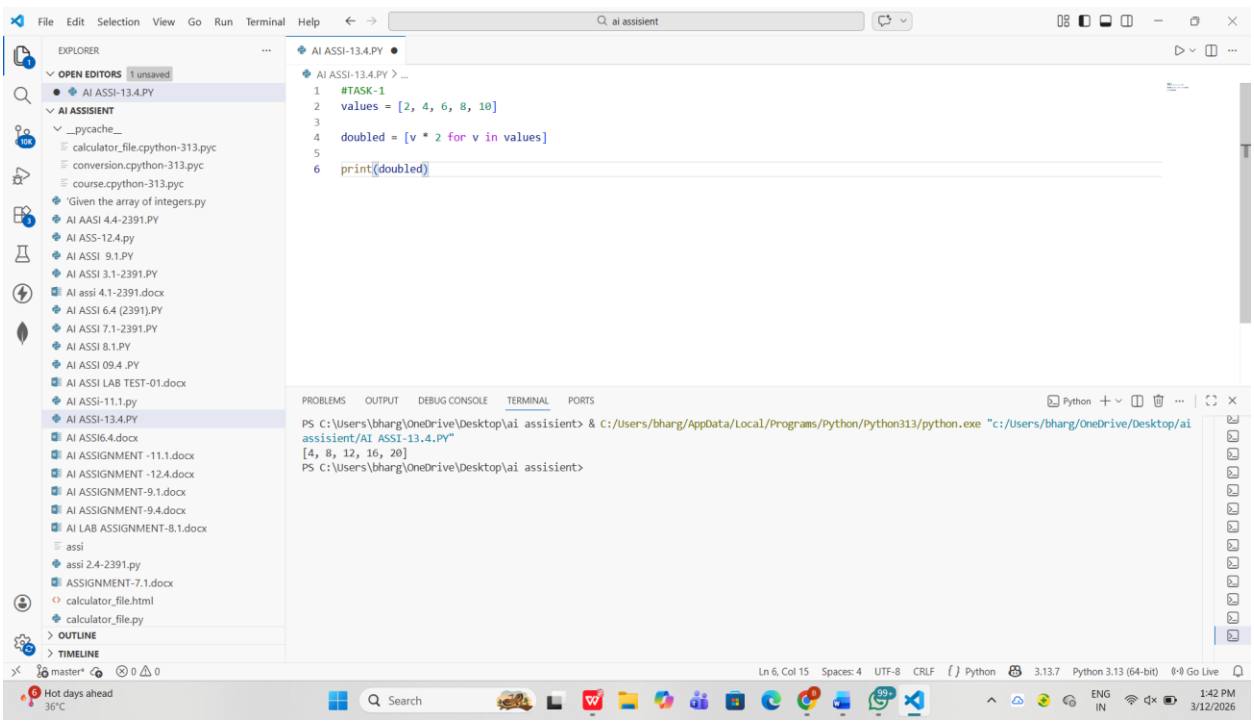
BATCH:44

Hall.no:2303A52405

Task 1: Refactoring Data TransformationLogic

Scenario:

You are maintaining a data preprocessing script where numerical transformations are written using verbose loops.



Observations:

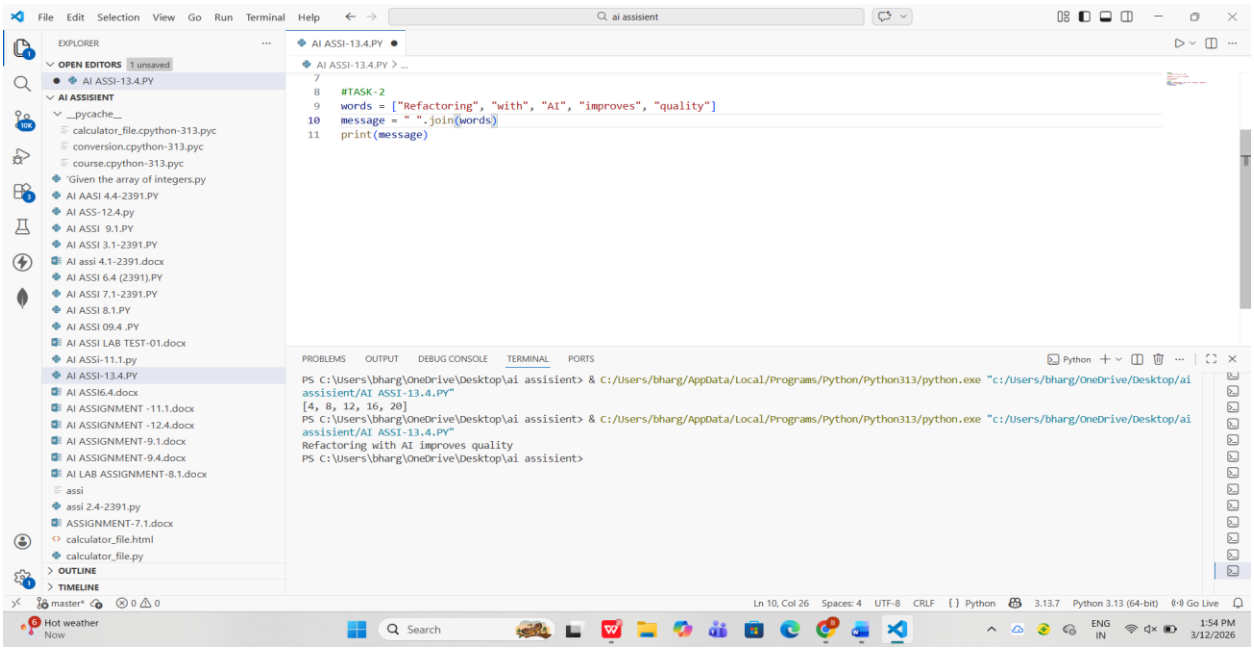
- The legacy code uses an **explicit loop** to iterate through each element.
- Values are transformed by **multiplying each number by 2**.

- A new list is created using the **append() method**.
- The code works correctly but is **longer and less Pythonic**.

Task 2: Improving Text Processing CodeReadability

Scenario:

You are working on a log message generator that builds messages word by word



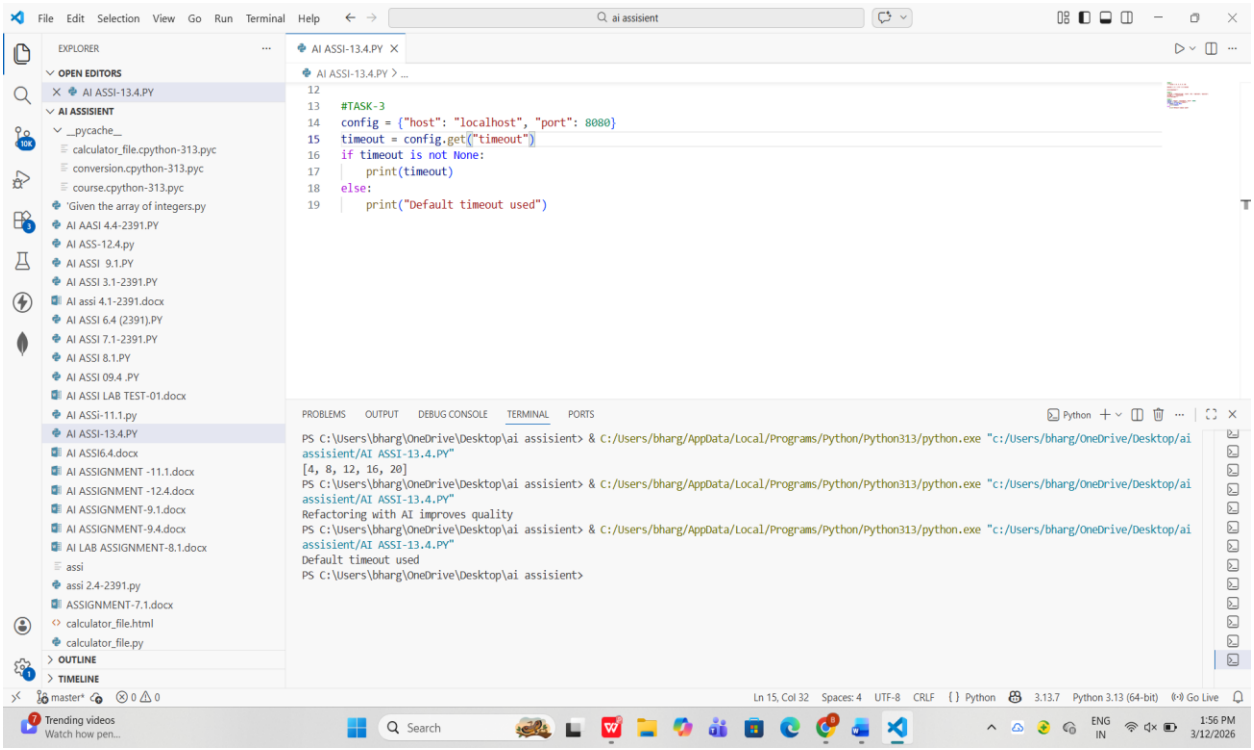
Observations:

- The legacy code builds a sentence using **repeated string concatenation (+=)**.
- Repeated concatenation inside loops is **less efficient in Python**.
- The code also requires **strip()** to remove the extra space at the end.
- Using **join() method** is a more efficient approach

Task 3: Safer Access to Configuration Data

Scenario:

You are maintaining a configuration loader where dictionary keys may or may not exist depending on deployment settings.



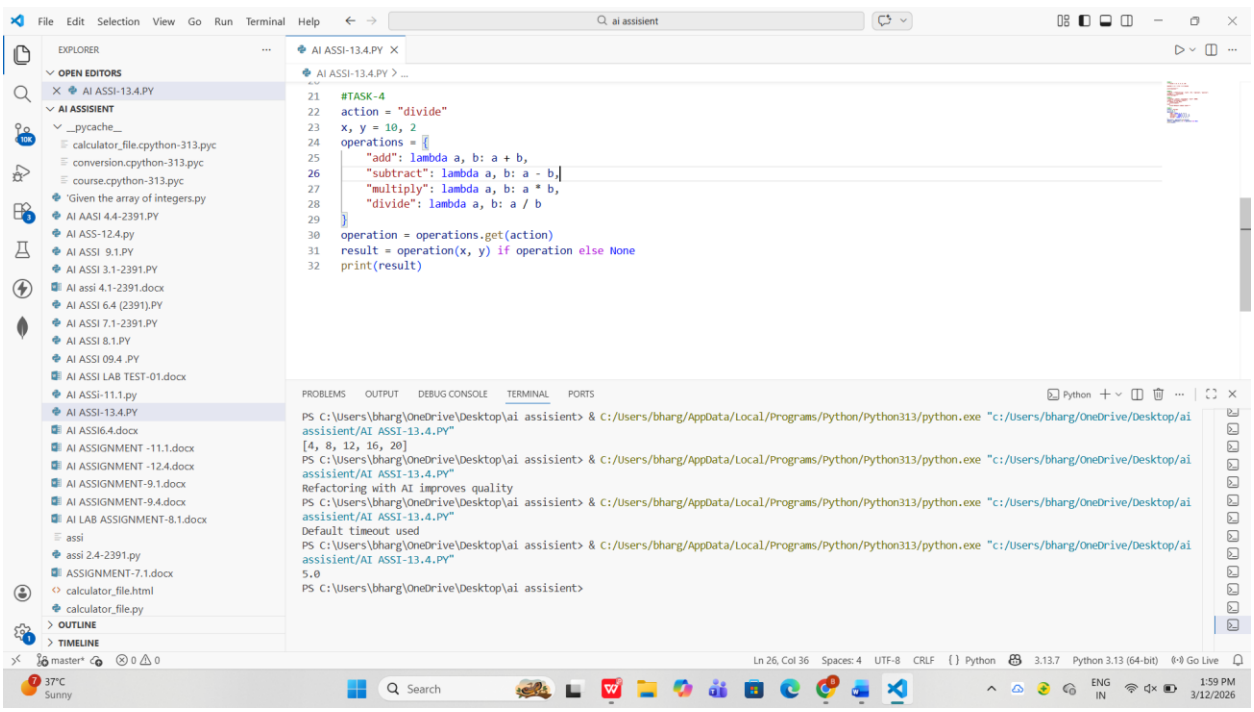
Observations:

- The legacy code checks dictionary keys using **if key in dictionary**.
- This approach requires **extra conditional logic**.
- Accessing missing keys directly can cause a **KeyError**.
- The **get() method** provides safer access to dictionary values.

Task 4: Refactoring Conditional Logic forScalability

Scenario:

You are enhancing a utility module that performs different operations based on user input.



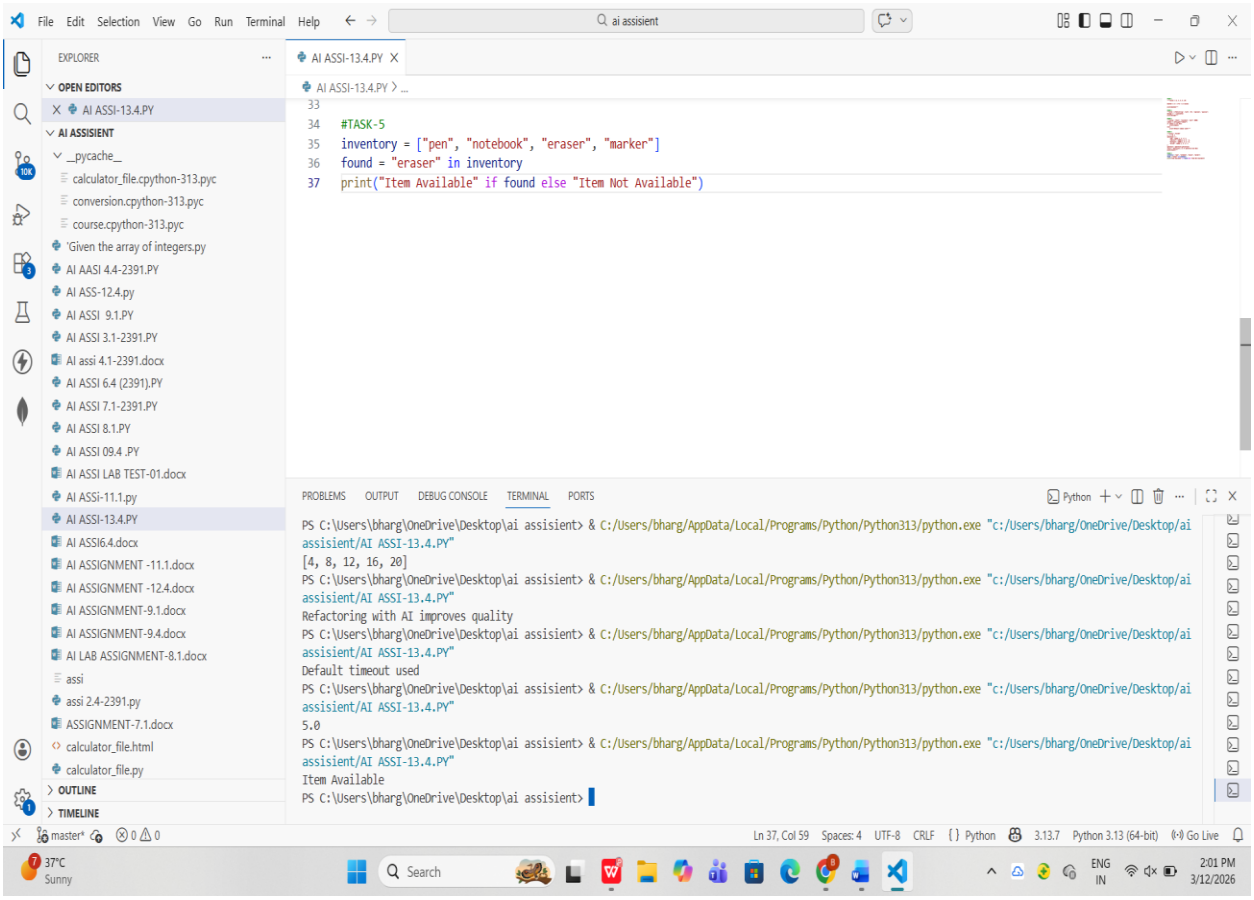
Observations:

- The legacy code uses **multiple if–elif conditions** to perform operations.
- As the number of operations increases, the code becomes **long and difficult to maintain**.
- A **dictionary mapping technique** can replace multiple conditions.
- Each operation is mapped to a **function or lambda expression**.

Task 5: Simplifying Search Logic inCollections

Scenario:

You are reviewing legacy inventory-check code that uses manual loops to find elements.



Observations:

- The legacy code searches for an item using a **manual loop**.
- A **flag variable (found)** is used to track whether the item exists.
- The loop uses **break** once the item is found.
- Python provides the **in operator** for membership testing.